

LeanMassCalculator

Android Application Engineering and Mobile Security Hardening

Mehdi Abouelmouahib

Project focus: native Android development in Kotlin, Firebase Authentication, Firestore synchronization, SQLite offline persistence, structured application architecture, and integrated mobile security controls inspired by OWASP MASVS.

Project Writeup

Contents

1	Executive Summary	4
2	Project Purpose	4
2.1	Problem Addressed	4
2.2	Formula Implemented	4
2.3	Result Interpretation	5
3	Functional Scope	5
3.1	Main Features	5
3.2	End-to-End User Flow	6
4	Technology Stack	6
4.1	Why Kotlin	6
4.2	Why Firebase	6
4.3	Why SQLite	7
5	Project Structure	7
5.1	Where Each Responsibility Lives	7
6	Application Architecture	8
6.1	Layered View	8
6.2	Navigation Architecture	9
7	Data Model	9
7.1	CalculationResult	9
7.2	Why This Model Matters	10
8	Authentication Pipeline	10
8.1	Firebase Authentication	10
8.2	Login Flow	10
8.3	Registration Flow	11
8.4	User Identity as a Data Boundary	11
9	Calculation Pipeline	11
9.1	Input Collection	11
9.2	Input Validation	12
9.3	Formula Selection	12
9.4	Visual Result Feedback	12

10 Local Persistence with SQLite	12
10.1 DatabaseHelper Role	12
10.2 SQLite Table	13
10.3 Insert Operation	13
10.4 Why Local Storage Is Useful	13
11 Cloud Persistence with Firestore	13
11.1 Firestore Data Organization	13
11.2 Cloud Synchronization Logic	14
11.3 Why UID-Based Isolation Matters	14
12 History Management	14
12.1 HistoryActivity	14
12.2 HistoryAdapter	14
12.3 Deletion Flow	15
13 Security Engineering	15
13.1 Security Design Philosophy	15
13.2 Security Surface Map	15
14 Security Implementation Details	16
14.1 AndroidManifest Hardening	16
14.2 Storage Protection	16
14.3 Network Security Configuration	17
14.4 Session Timeout	17
14.5 Generic Authentication Error Messages	18
14.6 Input Validation as Security and Quality Control	18
14.7 Root Detection Warning	18
15 Data Flow: From Input to History	19
16 Security Flow: From Login to Stored Data	19
17 Important Engineering Decisions	19
17.1 Using Firebase UID as the Main Ownership Key	20
17.2 Combining SQLite and Firestore	20
17.3 Keeping Security Lightweight	20
17.4 Separating Security Helpers	20
18 Engineering Challenges and Solutions	20

19 What the Project Demonstrates	21
19.1 Android Development Skills	21
19.2 Backend and Persistence Skills	21
19.3 Mobile Security Skills	21
20 Limitations	21
21 Future Improvements	22
22 Conclusion	22

Executive Summary

LeanMassCalculator is an Android application built in Kotlin to calculate and track a user's Lean Body Mass (LBM). The application combines a simple health-related feature with real mobile application engineering concerns: authentication, data persistence, offline usage, cloud synchronization, secure session handling, input validation, and protection of user-related data.

The user can create an account, log in, enter weight, height, and sex, obtain an LBM result using the Boer formula, and consult a history of previous calculations. Each result is stored locally in SQLite and remotely in Firebase Firestore under the authenticated user's identity. The project therefore goes beyond a basic calculator: it includes identity management, data modeling, local database design, cloud document organization, navigation, UI feedback, and security hardening.

Security is treated as part of the application design. The project addresses practical Android risks such as local data exposure, unclear network policy, long-lived sessions, weak input validation, verbose authentication errors, and execution on rooted devices. These protections are implemented through Android configuration, Kotlin utility code, and Activity-level integration.

One-line project pitch: LeanMassCalculator is a Kotlin Android application that combines Firebase authentication, SQLite and Firestore persistence, LBM health calculations, and integrated mobile security hardening.

Project Purpose

Problem Addressed

The application provides a simple way for a user to estimate and follow their Lean Body Mass. Lean Body Mass represents the portion of body mass that is not fat mass, including muscle, bones, organs, and water. Although the calculation itself is mathematically simple, the project uses it as a realistic base to build a complete mobile application around user accounts, stored history, and sensitive personal data.

The engineering challenge is not only to compute a value. The real goal is to build a complete application pipeline:

- collect user input safely;
- authenticate the user;
- calculate and display a meaningful result;
- store the result locally for offline access;
- synchronize the result with a cloud database;
- isolate each user's data;
- allow history consultation and deletion;
- protect the application against common mobile security weaknesses.

Formula Implemented

The calculation uses the Boer formula. The formula depends on the user's sex, weight, and height:

$$\text{LBM}_{\text{male}} = 0.407 \times \text{weight} + 0.267 \times \text{height} - 19.2$$

$$\text{LBM}_{\text{female}} = 0.252 \times \text{weight} + 0.473 \times \text{height} - 48.3$$

Weight is expressed in kilograms, height in centimeters, and the output is expressed in kilograms.

Result Interpretation

The app gives visual feedback after the calculation. The result is classified as satisfactory or to monitor according to simple thresholds:

Sex	Satisfactory threshold	Display behavior
Male	LBM greater than or equal to 38 kg	Satisfactory visual feedback
Female	LBM greater than or equal to 24 kg	Satisfactory visual feedback
Below threshold	Lower than the relevant threshold	Warning / monitoring feedback

Functional Scope

Main Features

The application includes the following functional modules:

Feature	Description
Registration	New users can create an account using email and password.
Login	Existing users can authenticate through Firebase Authentication.
Session access	Authenticated users are redirected to the calculator screen.
LBM calculation	The app calculates LBM from weight, height, and sex using the Boer formula.
Visual feedback	The result is displayed with a status indicating whether it is satisfactory or should be monitored.
Local history	Calculations are stored in SQLite and remain available offline.
Cloud persistence	Calculations are stored in Firestore under the authenticated user's UID.
History screen	The user can view previous calculations through a RecyclerView interface.
Deletion	The user can delete stored history entries.
Logout	The user can end the authenticated session.
Security controls	The app applies storage, network, authentication, validation, and resilience hardening.

End-to-End User Flow

The application flow is intentionally simple and understandable:

1. The user opens the application.
2. If the user is not authenticated, the login screen is displayed.
3. A new user can move to the registration screen and create an account.
4. After authentication, the user reaches the calculator screen.
5. The user enters weight, height, and sex.
6. The application validates the input.
7. The application computes the LBM result.
8. The result is displayed immediately with visual feedback.
9. The result is saved locally in SQLite.
10. The result is synchronized with Firestore.
11. The user can open the history screen to view previous records.
12. The user can delete records or log out.

Technology Stack

The application uses a native Android stack:

Category	Technology
Language	Kotlin
Mobile platform	Android, minimum SDK 24
User interface	XML layouts, Material Design 3, ViewBinding
Authentication	Firebase Authentication
Cloud database	Firebase Firestore
Local database	SQLite through SQLiteOpenHelper
Build system	Gradle
Application pattern	Activity-based structure with separated UI, data, model, and security responsibilities

Why Kotlin

Kotlin was used because it is the modern language for Android development. It provides concise syntax, null-safety, data classes, and smooth interoperability with the Android ecosystem. These features reduce boilerplate and make the code easier to maintain.

Why Firebase

Firebase Authentication avoids building a custom authentication backend. It provides account creation, login, logout, authenticated user state, and a unique user identifier. Firestore provides a cloud database that fits the document-based structure of user calculations.

Why SQLite

SQLite is used because it is embedded in Android and works offline. It allows the app to keep a local calculation history even when the device has no internet connection. This makes the project more realistic because it combines local-first behavior with cloud synchronization.

Project Structure

The project is organized as a standard Android application. The security part is not hidden in a separate report; it lives directly inside the Android project through the manifest, resource configuration, utility code, and Activity integration.

```
LeanMassCalculator/  
  app/  
    build.gradle  
    src/main/  
      AndroidManifest.xml  
      java/com/example/leanmasscalculator/  
        data/  
          DatabaseHelper.kt  
        model/  
          CalculationResult.kt  
        security/  
          SecurityUtils.kt  
        ui/  
          auth/  
            LoginActivity.kt  
            RegisterActivity.kt  
          calculator/  
            CalculatorActivity.kt  
          history/  
            HistoryActivity.kt  
            HistoryAdapter.kt  
        res/  
          drawable/  
          layout/  
          values/  
          xml/  
            network_security_config.xml  
    build.gradle  
    settings.gradle
```

Where Each Responsibility Lives

Component	Responsibility
AndroidManifest.xml	Declares app components, launcher Activity, exported status, backup policy, cleartext traffic policy, and network security configuration reference.
DatabaseHelper.kt	Creates and manages the SQLite database. Inserts, reads, updates, and deletes calculation records.
CalculationResult.kt	Represents one complete LBM calculation with local ID, Firestore ID, user ID, inputs, result, timestamp, and status.

Component	Responsibility
SecurityUtils.kt	Groups security-related helpers such as session timeout, timestamp tracking, session cleanup, and root detection.
LoginActivity.kt	Handles login, existing session checks, timeout verification, generic authentication errors, and navigation to the calculator.
RegisterActivity.kt	Handles account creation and local validation before calling Firebase Authentication.
CalculatorActivity.kt	Handles LBM input, validation, calculation, visual feedback, local storage, cloud synchronization, and activity timestamp refresh.
HistoryActivity.kt	Loads previous calculations for the authenticated user and manages history deletion.
HistoryAdapter.kt	Displays each calculation item inside a RecyclerView.
res/layout/	Contains XML screens for login, registration, calculator, history, and history items.
res/drawable/	Contains visual resources such as status icons and UI backgrounds.
res/values/	Contains colors, strings, and themes.
res/xml/network_security_config.xml	Defines HTTPS-only network policy and certificate trust anchors.

Important point: the security part is distributed across the application structure. It is not one isolated screen. It is implemented in the manifest, in **res/xml/**, in **security/SecurityUtils.kt**, and inside the authentication and calculator flows.

Application Architecture

Layered View

The application follows a simple layered organization:

```
User Interface Layer
  LoginActivity
  RegisterActivity
  CalculatorActivity
  HistoryActivity
  HistoryAdapter

Application Logic Layer
  Authentication flow
  Boer formula
  Input validation
  Session timeout checks
  Result interpretation

Data Layer
  CalculationResult
  DatabaseHelper
  Firebase Firestore collections
```

```
Security Layer
  AndroidManifest hardening
  network_security_config.xml
  SecurityUtils
  Generic authentication errors
  Root detection warning
```

Navigation Architecture

The navigation flow can be represented as follows:

```
LoginActivity
  |-- go to RegisterActivity
  |-- login success -> CalculatorActivity

RegisterActivity
  |-- registration success -> CalculatorActivity

CalculatorActivity
  |-- calculate LBM
  |-- save locally and remotely
  |-- open HistoryActivity
  |-- logout -> LoginActivity

HistoryActivity
  |-- display calculation history
  |-- delete selected calculation
  |-- return to CalculatorActivity
```

This structure keeps the app understandable. Each screen has a clear responsibility and the user journey remains linear.

Data Model

CalculationResult

The core model is **CalculationResult**. It represents a complete calculation record. The model is used by the calculator screen, SQLite helper, Firestore persistence, and history adapter.

A typical model contains:

- **id**: local SQLite identifier;
- **firestoreId**: remote Firestore document identifier;
- **userId**: Firebase UID of the authenticated user;
- **weight**: input weight in kilograms;
- **height**: input height in centimeters;
- **sex**: selected sex used to choose the formula;
- **lbm**: calculated Lean Body Mass;
- **date**: timestamp of the calculation;

- `isSatisfactory`: interpretation of the result.

A simplified representation is:

```
data class CalculationResult(  
    val id: Long = 0,  
    val firestoreId: String = "",  
    val userId: String = "",  
    val weight: Double = 0.0,  
    val height: Double = 0.0,  
    val sex: String = "",  
    val lbm: Double = 0.0,  
    val date: Long = System.currentTimeMillis(),  
    val isSatisfactory: Boolean = false  
)
```

Why This Model Matters

The model is not just a data container. It defines the contract between the different parts of the application:

- the calculator creates a new `CalculationResult`;
- SQLite stores and reads the object locally;
- Firestore stores it remotely;
- the history screen displays it;
- deletion uses the local ID and sometimes the Firestore ID;
- user isolation depends on the `userId` field.

Authentication Pipeline

Firestore Authentication

Authentication is handled with Firebase Authentication using email and password. The application does not manage raw credentials manually. Firebase handles account creation, login, session state, and user identity.

The key authentication operations are:

- `createUserWithEmailAndPassword(email, password)` for registration;
- `signInWithEmailAndPassword(email, password)` for login;
- `auth.currentUser` to check whether a user is already authenticated;
- `auth.signOut()` to log out;
- `currentUser.uid` to isolate local and cloud data.

Login Flow

The login screen performs four jobs:

1. initialize Firebase Authentication;
2. check whether a user session already exists;
3. validate that email and password fields are not empty;
4. call Firebase and navigate to the calculator after successful login.

A simplified login flow is:

```
private fun login() {  
    val email = binding.etEmail.text.toString().trim()  
    val password = binding.etPassword.text.toString().trim()  
  
    if (email.isEmpty() || password.isEmpty()) {  
        showMessage("Fill all fields")  
        return  
    }  
  
    auth.signInWithEmailAndPassword(email, password)  
        .addOnSuccessListener { goToCalculator() }  
        .addOnFailureListener { showMessage("Incorrect credentials.") }  
}
```

Registration Flow

The registration screen creates a new Firebase account. Before calling Firebase, the app validates local fields:

- email must not be empty;
- password must not be empty;
- password confirmation must match;
- password length must satisfy Firebase's minimum requirement.

This avoids unnecessary network calls and gives immediate feedback to the user.

User Identity as a Data Boundary

The Firebase UID is central to the design. It is used to scope data locally and remotely. Without this UID, calculation records from multiple users on the same device could be mixed together. With the UID, every stored result can be attached to the authenticated user.

Calculation Pipeline

Input Collection

The calculator screen collects three inputs:

- weight;
- height;
- sex.

The UI uses Material components and ViewBinding. ViewBinding gives typed access to views and avoids fragile calls to `findViewById`.

Input Validation

Before calculating the result, the app checks that inputs are present, numeric, and realistic. This validation is part of both quality and security: stored health-related records should not contain impossible values.

The hardened validation logic rejects unrealistic physiological values. For example:

```
if (weight < 1.0 || weight > 500.0) {
    showMessage("Invalid weight")
    return
}

if (height < 50.0 || height > 300.0) {
    showMessage("Invalid height")
    return
}
```

Formula Selection

The formula is selected based on sex:

```
val lbm = if (sex == "H") {
    0.407 * weight + 0.267 * height - 19.2
} else {
    0.252 * weight + 0.473 * height - 48.3
}
```

Visual Result Feedback

After computing the result, the app determines whether it is satisfactory according to the selected threshold. The UI then shows a status message and a visual indicator. This makes the result understandable without requiring the user to interpret the raw number alone.

Local Persistence with SQLite

DatabaseHelper Role

`DatabaseHelper.kt` manages the local SQLite database. It is responsible for:

- creating the database table;
- inserting new calculations;
- retrieving calculations for the authenticated user;
- updating the Firestore ID after synchronization;
- deleting a calculation from local storage.

SQLite Table

The local table stores all fields needed to reconstruct a calculation history entry.

Column	Type	Purpose
_id	INTEGER	Local primary key generated by SQLite.
user_id	TEXT	Firebase UID used to separate records per user.
weight	REAL	User weight in kilograms.
height	REAL	User height in centimeters.
sex	TEXT	Sex selected for the formula.
lbm	REAL	Calculated Lean Body Mass.
date	INTEGER	Timestamp of the calculation.
is_satisfactory	INTEGER	Boolean stored as 1 or 0.
firestore_id	TEXT	Cloud document identifier after Firestore synchronization.

Insert Operation

A simplified insert operation converts the model into `ContentValues` and inserts it into the local table:

```
fun insert(result: CalculationResult): Long {
    val values = ContentValues().apply {
        put("user_id", result.userId)
        put("weight", result.weight)
        put("height", result.height)
        put("sex", result.sex)
        put("lbm", result.lbm)
        put("date", result.date)
        put("is_satisfactory", if (result.isSatisfactory) 1 else 0)
        put("firestore_id", result.firestoreId)
    }
    return writableDatabase.insert("calculations", null, values)
}
```

Why Local Storage Is Useful

SQLite gives the project offline capability. The user can calculate and consult previous results even without a network connection. It also forces the design to consider data modeling, schema design, record ownership, and deletion behavior.

Cloud Persistence with Firestore

Firestore Data Organization

Firestore stores calculations under each authenticated user. The structure is:

```
users/  
  {uid}/  
    calculations/  
      {calculationId}  
        weight  
        height  
        sex  
        lbm  
        date  
        isSatisfactory
```

Cloud Synchronization Logic

After a calculation is created, it is stored locally and then synchronized remotely. The local record can keep the Firestore document ID, allowing future operations to link a local row with a cloud document.

A simplified persistence sequence is:

1. Create `CalculationResult` in `CalculatorActivity`.
2. Insert the result into SQLite.
3. Send the result to Firestore under `users/uid/calculations`.
4. Store the Firestore document ID locally when synchronization succeeds.

Why UID-Based Isolation Matters

User-specific paths prevent data from being mixed at the application level. Every calculation belongs to a Firebase UID. The same principle is used locally by storing `user_id` in SQLite and remotely by storing documents under `users/uid`.

History Management

HistoryActivity

The history screen displays previous calculations. It loads records belonging to the current authenticated user and passes them to a RecyclerView adapter.

The screen provides a simple but important feature: it proves that results are not only calculated once but persist as structured records.

HistoryAdapter

`HistoryAdapter.kt` is responsible for binding each `CalculationResult` to the UI. Each row can display:

- the calculation date;
- the input values;
- the LBM result;

- the status;
- a delete action.

Deletion Flow

Deletion is part of data lifecycle management. A calculation should not be considered permanent if the user decides to remove it. The deletion flow removes the local record and can also be tied to the Firestore document through the stored `firestoreId`.

Security Engineering

Security Design Philosophy

The application handles personal health-related input. This makes security relevant even if the application is small. The security work focuses on practical Android risks:

- local database extraction;
- weak or implicit network configuration;
- long-lived authenticated sessions;
- account enumeration through verbose errors;
- invalid data entering the database;
- execution on rooted or tampered devices.

The goal is not to claim the application is unbreakable. The goal is to show that the application was designed with realistic mobile risks in mind and that mitigations were integrated into the codebase.

Security Surface Map

Area	Risk	Control
Storage	SQLite data could be extracted through backup mechanisms.	Disable Android backup with <code>allowBackup="false"</code> .
Network	Cleartext traffic or weak certificate trust could expose data in transit.	Use <code>network_security_config.xml</code> ; disable cleartext traffic; trust system CAs.
Authentication	Existing Firebase session could remain accessible for too long.	Add inactivity timeout through <code>SecurityUtils.kt</code> .
Input handling	Unrealistic values could lead to bad calculations and stored invalid records.	Enforce physiological input bounds before calculation.
Error handling	Raw Firebase errors could reveal whether an account exists.	Replace verbose errors with generic authentication messages.

Area	Risk	Control
Runtime environment	Rooted devices can expose app data or allow tampering.	Add root detection warning to inform the user.
Component exposure	Activities could be reachable from other apps if exported incorrectly.	Keep non-launcher Activities with <code>exported="false"</code> .

Security Implementation Details

AndroidManifest Hardening

The manifest is a central security file because it declares application-level behavior. The app hardens the manifest with several choices:

- disable Android backup;
- disable cleartext traffic;
- reference the network security configuration;
- keep non-launcher Activities non-exported.

Example manifest hardening:

```
<application
    android:allowBackup="false"
    android:networkSecurityConfig="@xml/network_security_config"
    android:usesCleartextTraffic="false"
    android:theme="@style/Theme.LeanMassCalculator">

    <activity
        android:name=".ui.auth.LoginActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <activity android:name=".ui.auth.RegisterActivity" android:exported="false" />
    <activity android:name=".ui.calculator.CalculatorActivity" android:exported="false" />

    <activity android:name=".ui.history.HistoryActivity" android:exported="false" />
</application>
```

Storage Protection

The app stores user-related calculation data in SQLite. If Android backup is enabled, local data can be more easily extracted through device backup mechanisms. The application reduces that risk with:

```
android:allowBackup="false"
```

This is not full database encryption, but it is an important baseline control.

Network Security Configuration

The app adds a dedicated file under `res/xml/`:

```
res/xml/network_security_config.xml
```

The configuration is:

```
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system" />
    </trust-anchors>
  </base-config>
</network-security-config>
```

This explicitly blocks cleartext traffic and avoids trusting user-installed certificate authorities. It improves the clarity of the network policy and reduces the risk of accidental insecure communication.

Session Timeout

A Firebase session can remain active. This is convenient, but in a mobile context it creates a risk if a phone is unlocked and left unattended. The application adds a 30-minute timeout.

The timeout logic is placed in `SecurityUtils.kt`:

```
object SecurityUtils {
    private const val SESSION_TIMEOUT_MS = 30 * 60 * 1000L
    private const val PREF_NAME = "secure_session"
    private const val KEY_LAST_ACTIVE = "last_active_ts"

    fun recordActivity(context: Context) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit()
            .putLong(KEY_LAST_ACTIVE, System.currentTimeMillis())
            .apply()
    }

    fun isSessionExpired(context: Context): Boolean {
        val lastActive = context
            .getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .getLong(KEY_LAST_ACTIVE, 0L)

        if (lastActive == 0L) return false
        return System.currentTimeMillis() - lastActive > SESSION_TIMEOUT_MS
    }

    fun clearSession(context: Context) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit()
            .clear()
            .apply()
    }
}
```

The login flow then checks whether the current Firebase user is still allowed to continue:

```
if (auth.currentUser != null && !SecurityUtils.isSessionExpired(this)) {
    SecurityUtils.recordActivity(this)
}
```

```
        goToCalculator()
        return
    }

    if (auth.currentUser != null && SecurityUtils.isSessionExpired(this)) {
        auth.signOut()
        SecurityUtils.clearSession(this)
        showMessage("Session expired. Please log in again.")
    }
}
```

Generic Authentication Error Messages

Raw Firebase errors can provide useful information to attackers. For example, different messages may reveal whether an account exists. The app avoids exposing raw exception messages during login.

Instead of:

```
Toast.makeText(this, "Error: ${e.message}", Toast.LENGTH_LONG).show()
```

The app uses a generic message:

```
Toast.makeText(this, "Incorrect credentials.", Toast.LENGTH_LONG).show()
```

This is a small change, but it improves authentication privacy and reduces account enumeration risk.

Input Validation as Security and Quality Control

Input validation protects both the user experience and the stored data. The calculator rejects values that are technically numeric but physiologically impossible.

Example checks:

```
if (weight < 1.0 || weight > 500.0) {
    Toast.makeText(this, "Invalid weight", Toast.LENGTH_SHORT).show()
    return
}

if (height < 50.0 || height > 300.0) {
    Toast.makeText(this, "Invalid height", Toast.LENGTH_SHORT).show()
    return
}
```

This prevents meaningless LBM values from being displayed or stored.

Root Detection Warning

Rooted devices weaken Android's isolation model. An attacker with root access may inspect local files, hook runtime behavior, or modify app execution. The application includes a basic root detection helper.

A simplified version is:

```
private val ROOT_INDICATORS = arrayOf(
    "/system/app/Superuser.apk",
    "/system/app/SuperSU.apk",
    "/sbin/su",

```

```
"/system/bin/su",
"/system/xbin/su",
"/data/local/xbin/su",
"/data/local/bin/su"
)

fun isDeviceRooted(): Boolean {
    return ROOT_INDICATORS.any { File(it).exists() }
}
```

Root detection is not a perfect defense because it can be bypassed. In this application, it is used as a warning mechanism to inform the user that the environment is risky.

Data Flow: From Input to History

The complete application data flow can be summarized as follows:

```
User input
-> validation
-> Boer formula
-> visual feedback
-> CalculationResult object
-> SQLite insert
-> Firestore add
-> local record updated with Firestore ID
-> history screen loads records by Firebase UID
```

This pipeline connects UI, logic, persistence, cloud synchronization, and user identity.

Security Flow: From Login to Stored Data

The security flow is integrated into the normal application lifecycle:

```
Application launch
-> AndroidManifest policy applies
-> network policy is enforced
-> LoginActivity checks Firebase user
-> SecurityUtils checks session timeout
-> expired session signs out
-> successful login records activity timestamp
-> CalculatorActivity validates inputs
-> result stored under authenticated user ID
-> HistoryActivity reads only current user's records
-> root detection warns on risky environments
```

This is the core engineering value of the project: security is not just a checklist, it is connected to how the application works.

Important Engineering Decisions

Using Firebase UID as the Main Ownership Key

The Firebase UID is used as the ownership key for records. This avoids using email addresses as database identifiers and makes data isolation cleaner.

Combining SQLite and Firestore

Using both SQLite and Firestore adds complexity, but it makes the application more realistic. SQLite gives offline history, while Firestore provides cloud persistence.

Keeping Security Lightweight

The security controls are practical and understandable. The app does not add heavy mechanisms that would make it hard to maintain. Instead, it focuses on high-value improvements: manifest hardening, network configuration, session timeout, generic errors, validation, and root warning.

Separating Security Helpers

`SecurityUtils.kt` makes security-related logic reusable. Instead of duplicating timeout or root checks inside every Activity, the application can call a central helper.

Engineering Challenges and Solutions

Challenge	Why it mattered	Solution
Authentication state	The app needs to know whether the user can access the calculator.	Firebase Authentication and session checks.
User-specific records	Multiple users may use the same device.	Store and query records by Firebase UID.
Offline support	Cloud-only storage would make history unavailable offline.	Use SQLite as a local persistence layer.
Cloud synchronization	Local records should also exist in the cloud.	Store records in Firestore under <code>users/uid/calculations</code> .
Data deletion	Users need control over saved results.	Add deletion from history and link local records to cloud IDs.
Input quality	Bad values create meaningless health results.	Apply physiological bounds before calculation.
Session risk	Existing sessions can remain accessible.	Add inactivity timeout and forced sign-out.
Network risk	The app should not rely on unclear defaults.	Add explicit network security configuration.
Runtime risk	Rooted devices weaken app isolation.	Add root detection warning.

What the Project Demonstrates

This project demonstrates the ability to build a complete Android application while thinking about security and maintainability.

Android Development Skills

- Kotlin programming;
- Activity-based Android architecture;
- XML layouts and Material Design components;
- ViewBinding usage;
- RecyclerView and adapter pattern;
- Gradle configuration;
- app resources, themes, colors, and drawables.

Backend and Persistence Skills

- Firebase Authentication integration;
- Firestore document modeling;
- SQLite schema design;
- local insert, read, update, and delete operations;
- synchronization between local and remote storage;
- use of Firebase UID for data ownership.

Mobile Security Skills

- Android manifest hardening;
- network security configuration;
- storage exposure reduction;
- session timeout design;
- account enumeration reduction;
- input validation;
- root detection awareness;
- security controls integrated into normal application flow.

Limitations

The application is solid as an engineering project, but it is not a production-grade health application. Important limitations remain:

- SQLite is not encrypted by default;

- root detection can be bypassed;
- Firebase rules need to be carefully configured in the backend;
- Firestore synchronization conflict handling can be improved;
- the app does not include biometric re-authentication;
- there are no automated tests for all security-sensitive behaviors;
- the LBM thresholds are simple indicators and not medical advice.

Future Improvements

The project can be improved in several directions:

- encrypt the local SQLite database with SQLCipher or Android Jetpack Security;
- add biometric re-authentication before opening sensitive history data;
- strengthen and document Firestore security rules;
- add Play Integrity API for stronger device integrity checks;
- add unit tests for formula correctness and validation rules;
- add instrumentation tests for authentication and navigation flows;
- improve synchronization when Firestore is unavailable;
- add a privacy screen explaining how user data is stored;
- improve UI responsiveness and accessibility;
- add export functionality for the user's own history.

Conclusion

LeanMassCalculator is a complete Android application that connects a concrete user feature with real engineering concerns. It includes user authentication, LBM calculation, visual feedback, local SQLite storage, Firestore cloud persistence, history management, and integrated mobile security controls.

The main value of the project is the full pipeline. It starts from a functional mobile application and extends into data modeling, user identity, offline storage, cloud synchronization, and security hardening. The security work is part of the application itself: the manifest is hardened, the network policy is explicit, sessions are time-limited, inputs are validated, authentication errors are controlled, and rooted environments are detected.

This project shows the ability to build an Android application that is not only functional, but also structured, explainable, and security-aware. It demonstrates practical mobile development skills and the capacity to reason about how user data should be handled inside a real application.